
Frontend arhitektura kakvu **AI** razumije

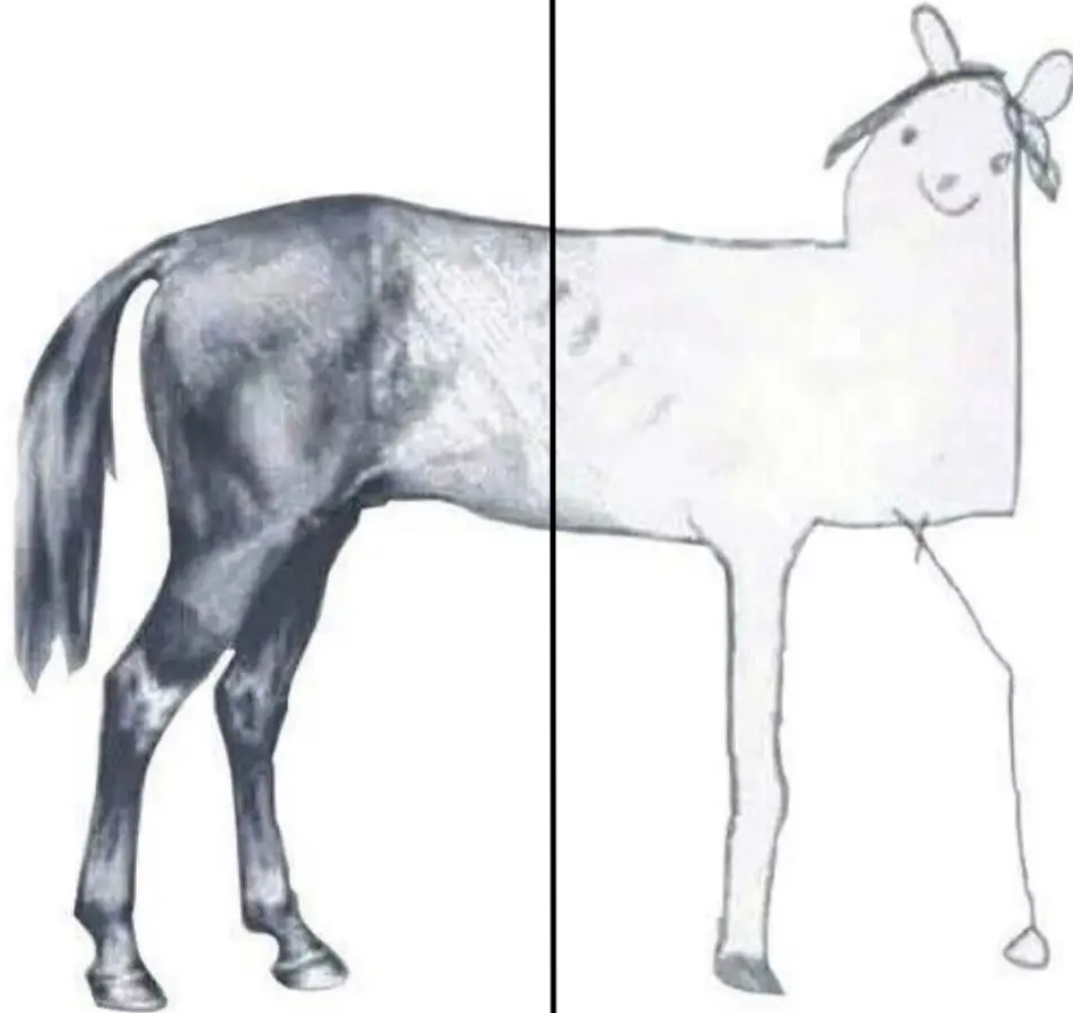
Koliko puta ste čuli ovu rečenicu?

“Napisao sam detaljni plan implementacije za Claude-a. Prvih par koraka - sve divno i krasno. Nakon toga, počeo se gubiti.”

OPUS 4.7 EXECUTING MY PLAN

STEP 1

STEP 2-10



Ili ovu rečenicu:

“Codex radi odlično na mojem mini projektu, ali je **beskoristan u našem produkcijskom kodu.”**

Gdje AI zapinje

AI asistenti su **stvarno sjajni** u pisanju koda.

Generiraj testove, napiši regex, složi boilerplate - mačji kašalj.

Gdje AI zapinje

AI asistenti su **stvarno sjajni** u pisanju koda.

Generiraj testove, napiši regex, složi boilerplate - mačji kašalj.

No, što je zadatak veći to AI-ju postaje **sve teže** pomoći:

Gdje AI zapinje

AI asistenti su **stvarno sjajni** u pisanju koda.

Generiraj testove, napiši regex, složi boilerplate - mačji kašalj.

No, što je zadatak veći to AI-ju postaje **sve teže** pomoći:

Razlog za to je jednostavan: **kontekst**.

Što je “**kontekst**” u LLM smislu?

Kontekst se odnosi na skup tokena koji su uključeni u proces generiranja (samplinga) iz LLM-a.

Što je “**kontekst**” u LLM smislu?

Kontekst se odnosi na skup tokena koji su uključeni u proces generiranja (samplinga) iz LLM-a.

Drugim riječima, sve što model može vidjeti kada generira odgovor - tvoj prompt, otvorene datoteke, struktura projekta itd.

Što je “**kontekst**” u LLM smislu?

Kontekst se odnosi na skup tokena koji su uključeni u proces generiranja (samplinga) iz LLM-a.

Drugim riječima, sve što model može vidjeti kada generira odgovor - tvoj prompt, otvorene datoteke, struktura projekta itd.

AI ne poznaje vaš projekt. On **čita samo ono što mu pružite** - i to je sve čime raspolaže.

Context Engineering

Context engineering = oblikovanje onoga što model vidi s ciljem dobivanja boljeg outputa

Context Engineering

Context engineering = oblikovanje onoga što model vidi s ciljem dobivanja boljeg outputa

Ne samo tvoj prompt - već i **programski kod**, **struktura direktorija**, **vanjski paketi**, **konvencije** itd.

Context Engineering

Context engineering = oblikovanje onoga što model vidi s ciljem dobivanja boljeg outputa

Ne samo tvoj prompt - već i **programski kod**, **struktura direktorija**, **vanjski paketi**, **konvencije** itd.

Kvaliteta odgovora **ograničena je** kvalitetom konteksta koji model prima.

Kako točno AI “čita” náš kod?

Pristup A: Embedding & Indexing

[Cursor](#)[Copilot](#)[Cody](#)

Korak 1. Kod otvaranja projekta, AI alat **indeksira tvoj codebase**:

Podijeli programski kod u chunkove



Generiraj embeddingse

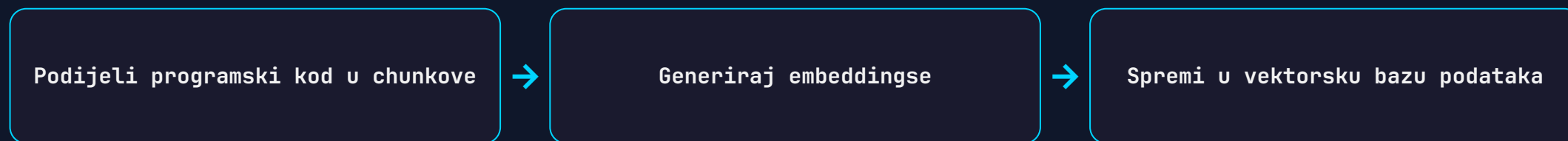


Spremi u vektorsku bazu podataka

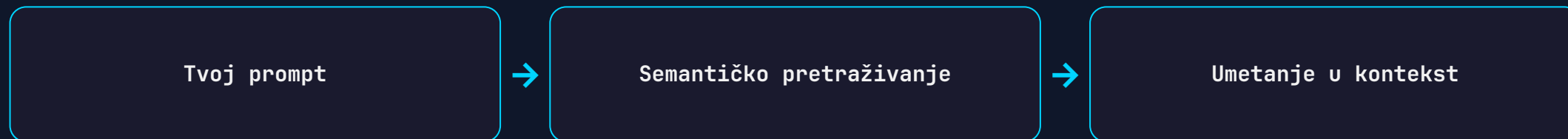
Pristup A: Embedding & Indexing

[Cursor](#)[Copilot](#)[Cody](#)

Korak 1. Kod otvaranja projekta, AI alat **indeksira tvoj codebase**:



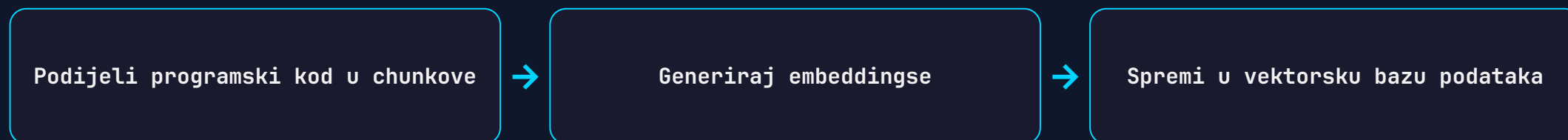
Step 2. Kod promptanja, **pretražuje relevantan kod**:



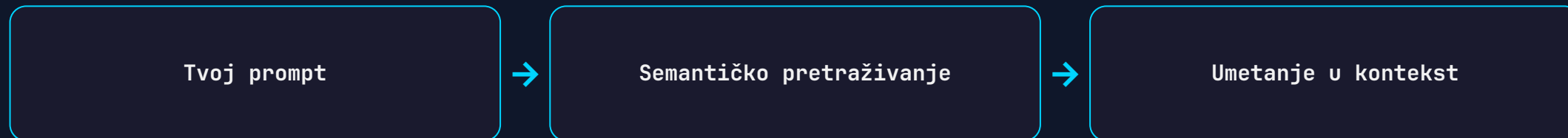
Pristup A: Embedding & Indexing

[Cursor](#)[Copilot](#)[Cody](#)

Korak 1. Kod otvaranja projekta, AI alat **indeksira tvoj codebase**:



Step 2. Kod promptanja, **pretražuje relevantan kod**:



Loša struktura → loši rezultati pretraživanja → loš output

Pristup B: Agentic Search

Claude Code

Nema indeksiranja. AI **istražuje po potrebi**, poput developera:

1. **Read file tree** → "Kako ovaj projekt izgleda?"
2. **Glob** (find files) → "Koje su datoteke vezane uz moj zadatak?"
3. **Grep** (search code) → "Koja od ovih komponenti sadrži ono što meni treba?"
4. **Read** (load file) → "Pokušavam shvatiti kako funkcionira ova komponenta."
5. **Follow imports** → "O čemu ovisi ova komponenta?"
6. Repeat 4-5 → **Svaki skok puni kontekst i troši tokene**

Pristup B: Agentic Search

Claude Code

Nema indeksiranja. AI **istražuje po potrebi**, poput developera:

1. **Read file tree** → "Kako ovaj projekt izgleda?"
2. **Glob** (find files) → "Koje su datoteke vezane uz moj zadatak?"
3. **Grep** (search code) → "Koja od ovih komponenti sadrži ono što meni treba?"
4. **Read** (load file) → "Pokušavam shvatiti kako funkcionira ova komponenta."
5. **Follow imports** → "O čemu ovisi ova komponenta?"
6. Repeat 4-5 → **Svaki skok puni kontekst i troši tokene**

Niska kohezija → brže punjenje konteksta → veća potrošnja tokena

Što je zajedničko?

- AI mora **odabrati** što će uključiti u kontekst
- **Imenovanje je bitno** - bolji embeddingsi, pattern matching
- **Jasne granice** = manje šuma, manje uzalud potrošenih tokena, bolji output

Gdje nastaju problemi

Prompt: "Dodaj delete button na profile card"

AI čita file tree i vidi:

```
src/  
  components/  
    Card.tsx           ← koji card? user? product? post?  
    CardSmall.tsx      ← da li je ovo isti card?  
    CardList.tsx       ← ili možda ovaj?  
    Button.tsx  
    Modal.tsx  
    Header.tsx  
    ... 194 more files  
  utils/  
    helpers.ts         ← 800 linija, 40 exporta, nema grupiranja  
  hooks/  
    useData.ts         ← dohvaća users? producte? oboje?  
  services/  
    api.ts             ← 3000 linija, svaki endpoint u jednom fileu
```

Problem 1: Nema semantičkog grupiranja. AI mora otvarati više različitih datoteka samo kako bi pronašao pravu karticu.

Problem 2: Generički nazivi poput `helpers.ts` i `useData.ts` ne pomažu pri pretraživanju

Problem 3: Monolitne datoteke (`api.ts`) prisiljavaju AI da učitava tisuće nepotrebnih linija koda u kontekst

Architecture = Context Engineering

Dobra arhitektura smanjuje količinu konteksta potrebnog AI-u da ispravno obavi svoj posao.

Architecture = Context Engineering

Dobra arhitektura smanjuje količinu konteksta potrebnog AI-u da ispravno obavi svoj posao.

Kako možemo izgraditi dobru arhitekturu?

Architecture = Context Engineering

Dobra arhitektura smanjuje količinu konteksta potrebnog AI-u da ispravno obavi svoj posao.

Kako možemo izgraditi dobru arhitekturu? Primjenom prave metodologije.

Feature-Sliced Design

Što je **Feature-Sliced Design**?

Arhitekturna metodologija za frontend projekte

FSD propisuje **kako organizirati kod** u projektu - gdje stavljati datoteke, kako ih grupirati i kako smiju ovisiti jedni o drugima.

Cilj: kod koji je **predvidljiv**, **izoliran** i **razumljiv iz strukture** - bez čitanja svake datoteke.

Drugim riječima - metodologija dizajnirana za isti problem koji AI ima: **izgraditi mentalni model projekta brzo i bez prethodnog konteksta.**



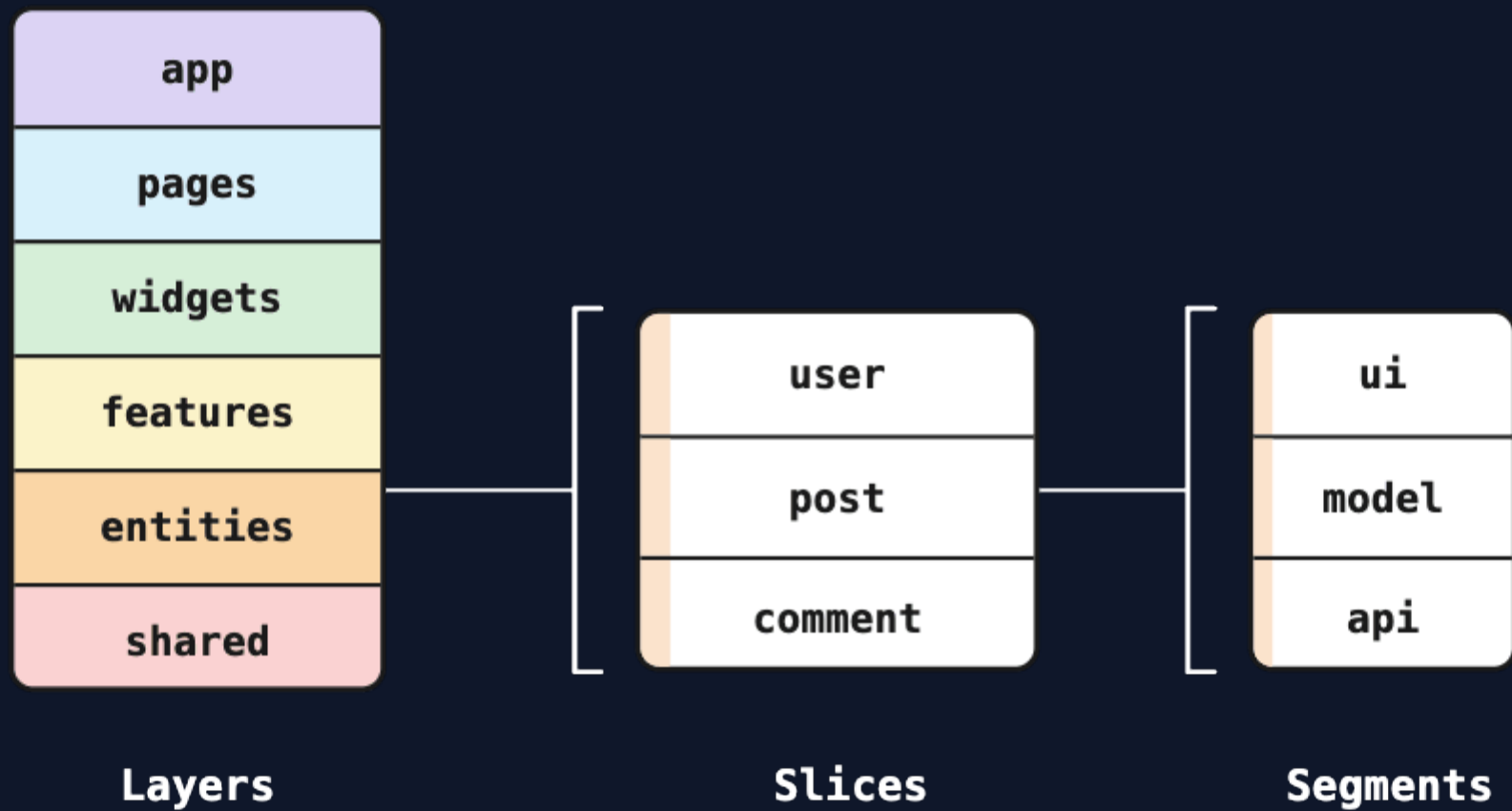
Layers



Layers

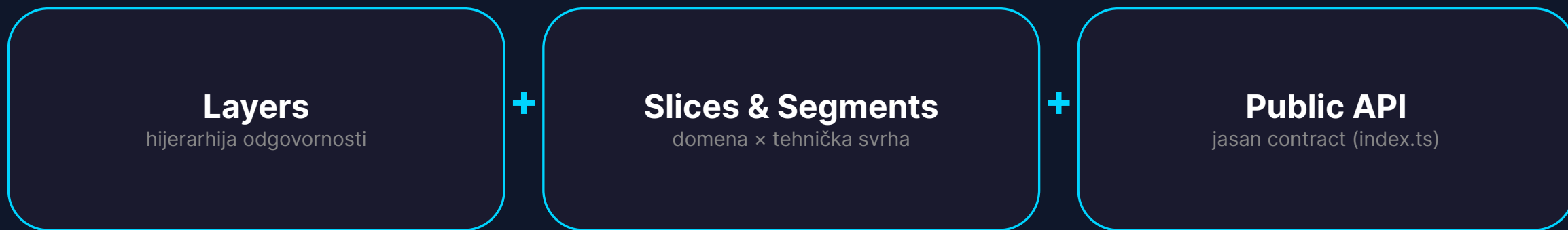


Slices



Tri stupa FSD-a

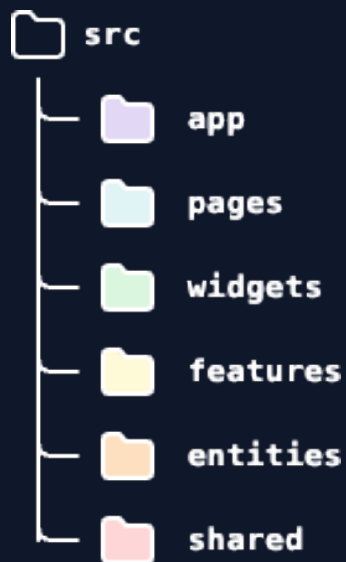
Sve se svodi na tri pravila



Svako od ova tri pravila **izravno smanjuje** količinu konteksta koji AI mora učitati da bi razumio projekt.

Pravilo #1: Slojevi

Hijerarhija odgovornosti, fiksni redoslijed



Mapa odgovornosti

AI pročitati file tree i **odmah zna** gdje je poslovna logika, gdje akcije, a gdje infrastruktura.

Pravilo importa

Sloj smije importati **samo iz slojeva ispod sebe**.
Cross-importi između sličica istog sloja su **zabranjeni**.



Manje context-hopova

Jednosmjerni tok importa = **konačan put kroz ovisnosti**. Manje koraka, manje potrošenih tokena.

Pravilo #2: Slices & Segments

Domena (slice) × tehnička svrha (segment)

Slice = poslovna domena

```
features/  
  auth/      ← jedan slice  
  profile-edit/ ← drugi slice  
  like-post/  ← treći slice
```

Slice grupira sve što pripada **jednoj domeni** — bez couplinga sa susjedima.

Segment = tehnička svrha

```
features/auth/  
  ui/      ← komponente, stilovi  
  model/    ← state, validacija  
  api/      ← backend pozivi  
  lib/      ← utility funkcije
```

Standardizirana imena segmenata **u svakom slice-u**.

Al pomoć: granice slice = manje context pollutiona, segmenti = **pattern matching** kroz cijeli projekt. „Trebam fetch korisnika?“ → entities/user/api/. Bez pretrage.

Pravilo #3: Public API

Svaki slice eksponira `index.ts` kao ugovor

```
features/auth/  
  ui/  
    LoginForm.tsx  
    LogoutButton.tsx  
  model/  
    auth-store.ts  
  api/  
    login.ts  
  index.ts ← jedini ulaz
```

```
// features/auth/index.ts  
export { LoginForm } from './ui/LoginForm'  
export { useAuth } from './model/auth-store'  
  
// Svugdje drugdje:  
import { LoginForm } from '@features/auth'  
  
// Zabranjeno:  
import { LoginForm } from '@features/auth/ui/LoginForm'
```

AI pročitati `index.ts` (8 redova) i zna kompletno sučelje slice-a — umjesto da učitati 400+ linija interne implementacije. **Štedi tokene, stabilan ugovor.**

AI alati za FSD

Daj AI-u FSD dokumentaciju kao kontekst — ne mora pogađati pravila.

LLMs.txt

`fsd.how/llms-full.txt`

Cijela FSD specifikacija u jednom plain-text fileu — formatirana baš tako da je AI agenti mogu lako progutati.

Doda se u `CLAUDE.md` / `AGENTS.md` ili kao referenca u promptu — AI dobiva pravila prije nego napiše ijednu liniju.

feature-sliced-design Skill

`/feature-sliced-design`

Službeni Claude Code skill — aktivira se automatski kad se radi o FSD strukturi (gdje datoteku staviti, public API, cross-import, segmenti).

Sadrži decision tree, anti-patterne i conditional reference fileove — **zna kada koje znanje učitati u kontekst.**

Dobra arhitektura + dobra dokumentacija u kontekstu = AI prestaje halucinirati i počinje pisati kod **po pravilima projekta.**

FSD × AI

Why Feature-Sliced Design is AI-friendly by nature

FSD Concept	AI Benefit
Layers app → shared	AI reads the file tree and immediately gets a responsibility map .
Import rule only import from below	One-directional dependency flow = minimal context hops.
Slices zero coupling	AI stays within feature boundaries — no context pollution .
Public API index.ts contracts	AI reads the interface, doesn't crawl internals — saves tokens.
Segments ui / api / model / lib	Standardized grouping = AI pattern-matches across all slices.